

National University of Computer and Emerging Sciences



DL-2001: Introduction to Data Science Lab Manual 08

Department of Data Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives	3
1 What is k-NN?	3
2 KNN from scratch.....	3
3 KNN using built-in libraries	3
3.1 Import Libraries	3
3.2 Load Data.....	4
3.3 Visualize the Data	4
3.4 Split Data into Training and Testing Sets	5
3.5 Initialize and Train the k-NN Model	5
3.6 Make Predictions on Test Data	5
3.7 Evaluate Using Confusion Matrix and Classification Report.....	6
3.8 Visualize the Decision Boundary.....	7

Objectives

After performing this lab, students shall be able to understand implementation of KNN:

- ✓ From scratch
- ✓ Using built-in libraries

1 What is k-NN?

k-Nearest Neighbors is a simple, instance-based machine learning algorithm used for both classification and regression tasks.

k-NN doesn't really have a training phase like other algorithms. Instead, it *stores* all the training data. When you want to classify or predict the output for a new data point (called a **query point**), the algorithm:

- Calculates the distance between the query point and all points in the training set. Common distance metrics include **Euclidean distance** (straight-line distance), Manhattan distance, or others depending on the problem.
- Selects the **k** closest points (neighbors) to the query point based on these distances.
- For **classification**: It looks at the classes of these k neighbors. The class that appears most frequently among the neighbors is assigned to the query point (majority voting).
- For **regression**: It takes the average (or weighted average) of the values of the k neighbors to predict a continuous output.

2 KNN from scratch

Partial from scratch implementation of kNN is provided in the attached notebook.

3 KNN using built-in libraries

A general flow of implementing KNN is given below:

3.1 Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

3.2 Load Data

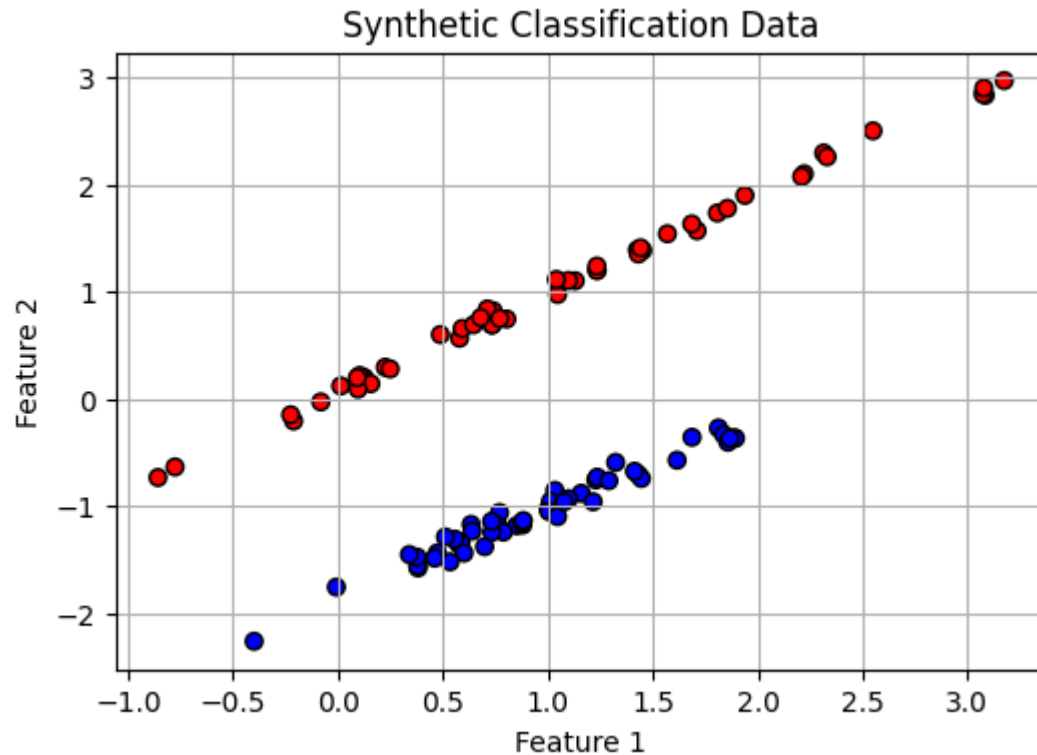
```
# Create a sample dataset
# Generate 2D classification data
X, y = make_classification(
    n_samples=100,    # number of samples
    n_features=2,     # number of features (must be 2 for plotting)
    n_redundant=0,    # no redundant features
    n_clusters_per_class=1,
    random_state=42
)
```

This example creates a sample synthetic dataset. Load your own dataset and set features and target columns as required

3.3 Visualize the Data

```
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='bwr', edgecolors='k')
plt.title("Generated Classification Data")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
```

The above visualization can only be done using 2D data



3.4 Split Data into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3.5 Initialize and Train the k-NN Model

```
# Initialize model with k = 3  
knn = KNeighborsClassifier(n_neighbors=3)  
# Train the model  
knn.fit(X_train, y_train)
```

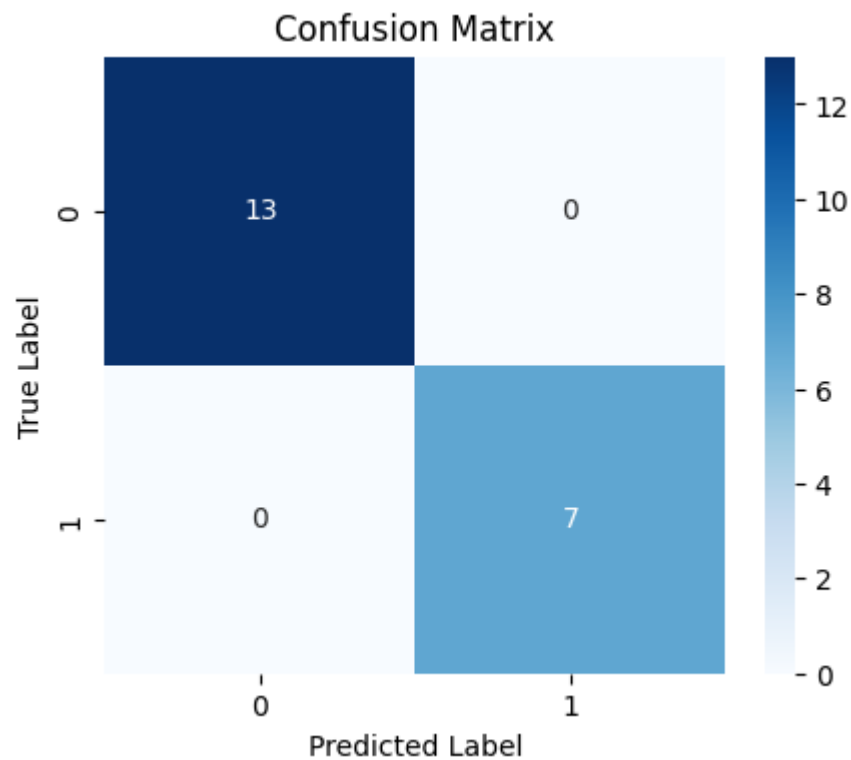
3.6 Make Predictions on Test Data

```
# Predict test labels  
y_pred = knn.predict(X_test)
```

3.7 Evaluate Using Confusion Matrix and Classification Report

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=[0, 1], yticklabels=[0, 1])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()

# Classification Report
cr = classification_report(y_test, y_pred)
print("\nClassification Report:")
print(cr)
```



```

Classification Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00        13
     1       1.00      1.00      1.00         7

 accuracy          1.00          1.00          1.00         20
 macro avg          1.00          1.00          1.00         20
 weighted avg          1.00          1.00          1.00         20

```

3.8 Visualize the Decision Boundary

```

# Create a mesh to plot decision boundaries
h = 0.1 # step size in the mesh
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

# Predict on mesh grid
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='bwr')
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap='bwr')
plt.title("k-NN Decision Boundary (k=3)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()

```

